SPECIFICATION

Calypso Terminal API

Calypso Crypto Legacy SAM

Version 2.0.0, 2026-10-28



Warning. The present document cancels and replaces the previous release.

© Calypso Networks Association 2026— Reproduction is prohibited without prior written agreement

Link and Contact



Website
https://calypsonet.org



Support
support@calypsonet.org

Copyright © Calypso Networks Association, https://calypsonet.org.

This specification is licensed under the **Creative Commons Attribution-NoDerivatives 4.0 International** *(CC BY-ND 4.0).*

You are free to **share** *— copy and redistribute this document in any medium or format for any purpose, even commercially — under the following terms:*

- ***Attribution*** *— You MUST give appropriate credit to the Calypso Networks Association, provide a link to the license, and indicate if changes were made. You MAY do so in any reasonable manner, but not in any way that suggests the Calypso Networks Association endorses you or your use.*
- ***NoDerivatives*** *— If you remix, transform, or build upon this document, you MAY NOT distribute the modified material.*

No additional restrictions — You MAY NOT apply legal terms or technological measures that legally restrict others from doing anything the license permits.

Implementation grant *— The* **NoDerivatives** *condition above applies to this specification* **document** *only — its text, tables and diagrams. It does* **not** *restrict the use of the technical information the document contains. The Calypso Networks Association hereby grants everyone an irrevocable, worldwide, royalty-free permission to use the information contained in this specification to design, develop, and distribute software implementations of this API, in any programming language, under the license of their choice.*

The full license text is available at https://creativecommons.org/licenses/by-nd/4.0/.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Table of Contents

1. Abstract . . . 6
2. Introduction . . . 7
2.1. Purpose . . . 7
2.2. Scope . . . 7
2.3. Conformance . . . 8
2.4. Document Conventions . . . 8
2.4.1. Naming . . . 8
2.4.2. Data types . . . 8
2.4.3. Type tables . . . 9
2.4.4. Operation tables . . . 9
2.4.5. Operation contracts . . . 9
2.4.6. Concurrency . . . 10
2.5. References and Resources . . . 10
2.5.1. Calypso References . . . 10
2.5.2. Normative References . . . 10
2.5.3. External Resources . . . 10
3. Glossary and Acronyms . . . 11
3.1. Glossary . . . 11
3.2. Acronyms . . . 11
4. Architectural Overview . . . 12
4.1. Functional positioning . . . 12
4.2. Logical namespaces . . . 12
4.3. UML class diagram . . . 12
5. API Specification . . . 14
5.1. Constants . . . 14
5.1.1. Legacy SAM API Properties . . . 14
5.2. Data . . . 14
5.2.1. Basic Signature Computation Data . . . 14
5.2.2. Basic Signature Verification Data . . . 14
5.2.3. Key Parameters . . . 15
5.2.4. Legacy Card Certificate Computation Data . . . 15
5.2.5. Legacy SAM Security Settings . . . 16
5.2.6. Traceable Signature Computation Data . . . 16
5.2.7. Traceable Signature Verification Data . . . 16
5.3. API Interfaces . . . 17
5.3.1. Async Transaction Creator Manager . . . 17
Export Commands . . . 17
5.3.2. Async Transaction Executor Manager . . . 18
5.3.3. Card Transaction Legacy SAM Extension . . . 18
Prepare Compute Signature . . . 18
Prepare Verify Signature . . . 19
5.3.4. Free Transaction Manager . . . 19
Export Target SAM Context For Async Transaction . . . 19
Prepare Compute Card Certificate . . . 20
Prepare Compute Signature . . . 20
Prepare Generate Card Asymmetric Key Pair . . . 20
Prepare Get Data . . . 21

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Prepare Plain Write Lock . . . 21
Prepare Verify Signature . . . 21
5.3.5. Legacy SAM . . . 22
Get Application Sub Type . . . 22
Get Application Type . . . 22
Get CA Certificate . . . 22
Get Computed Card Certificate . . . 23
Get Counter Ceilings . . . 23
Get Counter Increment Accesses . . . 23
Get Counters . . . 23
Get Key Pair . . . 24
Get Platform . . . 24
Get Product Info . . . 24
Get Product Type . . . 24
Get SAM Parameters . . . 25
Get Serial Number . . . 25
Get Signature . . . 25
Get Signed Data . . . 25
Get Software Issuer . . . 26
Get Software Revision . . . 26
Get Software Version . . . 26
Get System Key Parameters . . . 26
Get System Key Parameters Raw Data . . . 26
Get Work Key Parameters By KIF KVC . . . 27
Get Work Key Parameters By Record Number . . . 27
Get Work Key Parameters Raw Data By KIF KVC . . . 27
Get Work Key Parameters Raw Data By Record Number . . . 28
Is Signature Valid . . . 28
5.3.6. Legacy SAM API Factory . . . 28
Create Async Transaction Creator Manager . . . 29
Create Async Transaction Executor Manager . . . 29
Create Free Transaction Manager . . . 29
Create Legacy SAM Selection Extension . . . 30
Create Secure Write Transaction Manager . . . 30
Create Symmetric Crypto Card Transaction Manager Factory . . . 30
5.3.7. Legacy SAM Selection Extension . . . 30
Prepare Get Data . . . 31
Prepare Read All Counters Status . . . 31
Prepare Read Counter Status . . . 31
Prepare Read SAM Parameters . . . 32
Prepare Read System Key Parameters . . . 32
Prepare Read Work Key Parameters By KIF KVC . . . 32
Prepare Read Work Key Parameters By Record Number . . . 32
Set Dynamic Unlock Data Provider . . . 33
Set Dynamic Unlock Data Provider With Deferred Reader . . . 33
Set Static Unlock Data Provider . . . 33
Set Static Unlock Data Provider With Deferred Reader . . . 34
Set Unlock Data . . . 34
Set Unlock Data For Product Type . . . 34

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

5.3.8. Read Transaction Manager . . . 35
Prepare Read All Counters Status . . . 35
Prepare Read Counter Status . . . 35
Prepare Read SAM Parameters . . . 36
Prepare Read System Key Parameters . . . 36
Prepare Read Work Key Parameters By KIF KVC . . . 36
Prepare Read Work Key Parameters By Record Number . . . 36
5.3.9. Secure Write Transaction Manager . . . 37
Prepare Plain Write Lock . . . 37
Prepare Transfer Lock . . . 37
Prepare Transfer Lock Diversified . . . 38
Prepare Transfer System Key . . . 38
Prepare Transfer System Key Diversified . . . 38
Prepare Transfer Work Key . . . 39
Prepare Transfer Work Key Diversified . . . 39
Prepare Transfer Work Key Diversified With Specific Diversifier . . . 39
Prepare Write SAM Parameters . . . 40
5.3.10. Signature Computation Data . . . 40
5.3.11. Signature Verification Data . . . 40
5.3.12. Transaction Manager . . . 41
5.3.13. Write Transaction Manager . . . 41
Prepare Write Counter Ceiling . . . 41
Prepare Write Counter Configuration . . . 42
5.4. SPI Interfaces . . . 42
5.4.1. Legacy SAM Dynamic Unlock Data Provider SPI . . . 42
Get Unlock Data . . . 42
5.4.2. Legacy SAM Revocation Service SPI . . . 43
Is SAM Revoked . . . 43
5.4.3. Legacy SAM Static Unlock Data Provider SPI . . . 43
Get Unlock Data . . . 43
5.5. Enumerations . . . 44
5.5.1. Counter Increment Access . . . 44
5.5.2. Get Data Tag . . . 44
5.5.3. Legacy SAM Product Type . . . 44
5.5.4. SAM Traceability Mode . . . 44
5.5.5. System Key Type . . . 45
5.6. Errors . . . 45
5.6.1. Inconsistent Data . . . 45
5.6.2. Invalid Signature . . . 46
5.6.3. SAM Revoked . . . 46

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

1. Abstract

This document is the official technical specification of the **Terminal Calypso Crypto Legacy SAM API** standardised by the **Calypso Networks Association** (CNA). It defines, in a language-agnostic way (using a notation inspired by the Kotlin language), the interfaces, classes, enumerations, errors, structural relationships and behavioural requirements that any conforming implementation of the Terminal Calypso Crypto Legacy SAM API MUST satisfy.

The Terminal Calypso Crypto Legacy SAM API exposes the abstractions required to operate Calypso **Legacy SAM** products (SAM C1, HSM C1, S1E1, S1Dx). It is the symmetric counterpart of the **Terminal Calypso Crypto Symmetric API** (CNA-TCCS-API) for the specific case where the crypto module is a legacy Calypso SAM.

Document Status

Reference	261028-SP-CNATerminalAPI-CalypsoCryptoLegacySAM
Short name	CNA-TCCL-API
Version	2.0.0
Revision date	2026-10-28
Editor	Calypso Networks Association
Source repository	https://github.com/calypsonet/calypsonet-terminal-calypso-crypto-legacysam-api-spec
Reference license	Creative Commons Attribution-NoDerivatives 4.0 International (CC BY-ND 4.0)



The present document specifies the 2.0.0 of the Terminal Calypso Crypto Legacy SAM API. It defines a single, coherent baseline against which conforming implementations are evaluated; the **Since** column indicates the version in which each member was introduced.

Revision List



This section lists the high-level changes per version. The complete, fine-grained changelog is maintained in the `CHANGELOG.md` file at the root of the source repository.

Version / Date	Modifications
v2.0.0 2026-10-28	Initial edition of this document.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

2. Introduction

2.1. Purpose

The Terminal Calypso Crypto Legacy SAM API defines the public surface used by terminal applications and card-transaction managers to:

- select, unlock and identify a Legacy SAM;
- read SAM parameters, system keys, work keys and event counters;
- write counter ceilings and configurations (with or without a control SAM);
- compute and verify **basic** and **traceable** signatures using SAM keys;
- perform PKI-related operations on the SAM: key-pair generation, card-certificate computation, lock-file transfer/write;
- operate **asynchronous** write transactions through a create/execute split;
- enrich a Calypso card transaction with SAM-backed crypto operations through a card-transaction extension.

The API spans three categories of transaction managers:

Manager family	Purpose
`FreeTransactionManager`	Operations on the SAM without involving a **control SAM**—the typical "read" workflow plus signature computation and verification.
`SecureWriteTransactionManager`	Synchronous **write** operations on the SAM, requiring a **control SAM** to authorise the key/lock transfers.
`AsyncTransactionCreatorManager` / `AsyncTransactionExecutorManager`	Two-step **asynchronous** write workflow: the creator prepares the command stream using a control SAM, the executor replays it on the target SAM.

2.2. Scope

This specification covers:

- the SAM data model (`LegacySam`, `KeyParameters`);
- the selection extension used to enrich a Calypso reader selection with SAM-specific commands;
- the transaction managers, signature data carriers, key-pair and certificate computation data carriers;
- the security settings carrier and the card-transaction crypto extension;
- the SPIs implemented by the application to provide unlock data and revocation information;
- the errors raised by the transaction managers.

The following topics are **out of scope**:

- the on-the-wire APDU dialog with the SAM (handled internally by implementations),
- the algorithms used by the SAM to derive keys or sign data,
- the storage of certificates and key parameters outside the SAM.

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

2.3. Conformance

A software product conforms to this specification if and only if:

1. it implements every interface and class defined in Chapter 5 with the operations and semantics prescribed in this document,
2. it raises errors exclusively of the types defined in Section 5.6 for the situations described,
3. it preserves the structural relationships described in Chapter 4.

Throughout this specification, the key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY** and **OPTIONAL** are to be interpreted as described in RFC 2119 and RFC 8174 when, and only when, they appear in all capitals.

In conformance with RFC 2119, **SHALL** is equivalent to **MUST**; this specification uses **MUST** exclusively in order to avoid ambiguity.

2.4. Document Conventions

2.4.1. Naming

Type names follow `UpperCamelCase`; operation, parameter and enumeration-member names follow `lowerCamelCase` except for enumeration values which use `UPPER_SNAKE_CASE`. Names defined by this specification are reproduced as-is in the UML class diagram (Section 4.3).

2.4.2. Data types

The following type names, whose notation is inspired by the Kotlin language, are used in operation signatures and are mapped, in each binding, to the closest equivalent native type. Implementations MUST preserve the semantic intent rather than the syntactic form.

Type	Description	Typical bindings
`String`	Sequence of characters, UTF-8 encoded by default.	`String`, `string`, `str`.
`Boolean`	Two-state truth value.	`boolean`, `bool`.
`Byte`	Signed 8-bit value.	`byte`, `i8`.
`Int`	Signed 32-bit integer.	`int`, `Int32`, `i32`.
`ByteArray`	Ordered sequence of octets.	`byte[]`, `[u8]`, `bytes`.
`LocalDate`	Calendar date without a time-of-day or timezone.	`LocalDate`, `Date`, `NaiveDate`.
`SortedMap<K,V>`	Associative array sorted by key.	`SortedMap<K,V>`, `BTreeMap<K,V>`.
`Any`	Value of any type, root of the type hierarchy.	`Object`, `Any`, `object`, `Box<dyn Any>`.
`T?`	Nullable value: either a `T` value or `null`.	`Integer`/`Byte` (boxed), `Optional<T>`, `T?`, `Option<T>`.
`null`	Absence of value, the alternative held by a nullable type.	`null`, `nil`, `None`.
`Self`	Own type of the receiver: a fluent operation returns the instance it was called on.	`Self`, `T` (recursive generic).
`Unit`	Indicates that an operation returns no value.	`void`, `Unit`, `()`.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

2.4.3. Type tables

Each interface, class, enumeration and error defined in this document is described by a table of the form:

Name	The type's normalized name.
Kind	The category of the type (interface, class, enumeration, error, etc.).
Namespace	The namespace in which the type is defined.
Extends	The type extended by this type, when applicable.
Implements	The interface implemented by this type, when applicable.
Since	The version of the API in which the type was introduced.
Purpose	A normative description of the type's responsibility.
See also	Cross-references to related operations or types, each a hyperlink to the corresponding table. Present only when applicable.

The **Purpose** row states, in one or two sentences, the responsibility of the type and the way an instance is obtained. It is meant to be scanned, not read: any content that requires structure or depth—rules, lists, notes, value tables or examples—is placed as prose below the table.

2.4.4. Operation tables

Each operation defined in this document is described by a table of the form:

Signature	The operation's signature.
Since	The version of the API in which the operation was introduced.
Description	A normative description of the operation's behaviour.
Parameters	A description of each input parameter.
Returns	A description of the returned value, if any.
Pre-conditions	The conditions the caller MUST satisfy before invoking the operation, each prefixed by its nature.
Errors	A list of error conditions a correct caller must handle, each mapped to a specific error type.
See also	Cross-references to related operations or types, each a hyperlink to the corresponding table. Present only when applicable.

2.4.5. Operation contracts

To avoid repetition, the following rules apply to every operation table in Chapter 5 and are therefore not restated for each operation:

- **Pre-conditions.** The **Pre-conditions** row states what the caller MUST guarantee before invoking the operation. Violating one is a contract error, never listed in the **Errors** row, which describes only the outcomes a correct caller must handle. Each entry is prefixed by its nature, which determines the error raised when the condition is not met:
  - **Argument**—an illegal argument error. Applies to any invalid argument, including values bounded by a card or SAM protocol.
  - **Range**—an index out of bounds error. Applies when the argument designates a position within a collection or memory image exposed by the API.
  - **State**—an illegal state error.
  - **Capability**—an unsupported operation error.
- **Non-`null` results.** Unless the return type is marked nullable (`T?`) or the **Returns** row states otherwise, an operation returns a non-`null` value.

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

- **Collections.** Unless the return type is marked nullable (`T?`), an operation whose return type is a collection (e.g. `List`, `Set`, `Map`) returns an empty collection rather than `null`.
- **Fluent composition.** Builder-style operations return the current instance so that calls can be chained.

2.4.6. Concurrency

Unless explicitly stated otherwise, the stateful types defined by this specification are **not** thread-safe. A given instance MUST be accessed from a single thread at a time; concurrent use of the same instance without external synchronisation results in undefined behaviour. Distinct instances MAY be used concurrently.

2.5. References and Resources

2.5.1. Calypso References

References to CNA Terminal APIs designate the indicated **major** version; any later release within the same major is backward-compatible per that API's evolution policy.

Reference	Document
[CNA-TR-API]	*CNA Terminal API — Reader* (SP-CNATerminalAPI-Reader), version 3.0, Calypso Networks Association.
[CNA-TCC-API]	*CNA Terminal API — Calypso Card* (SP-CNATerminalAPI-CalypsoCard), version 3.0, Calypso Networks Association.
[CNA-TCCS-API]	*CNA Terminal API — Calypso Crypto Symmetric* (SP-CNATerminalAPI-CalypsoCryptoSymmetric), version 0.1, Calypso Networks Association.

2.5.2. Normative References

Reference	Document
[RFC 2119]	*Key words for use in RFCs to Indicate Requirement Levels*, S. Bradner, March 1997.
[RFC 8174]	*Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*, B. Leiba, May 2017.
[ISO/IEC 7816]	*Identification cards — Integrated circuit cards*.

2.5.3. External Resources

Resource	Link
Calypso Networks Association	https://calypsonet.org
Terminal APIs website	https://terminal-api.calypsonet.org
Terminal APIs documentation	https://docs.terminal-api.calypsonet.org

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

3. Glossary and Acronyms

3.1. Glossary

Term	Definition
SAM	Secure Application Module—a tamper-resistant cryptographic module used to perform Calypso operations.
Legacy SAM	A SAM compliant with the original Calypso SAM specifications (SAM C1, HSM C1, S1E1, S1Dx).
Control SAM	SAM used to authorise sensitive operations on a **target SAM** (e.g. key transfer).
Target SAM	SAM on which the operations are applied.
System Key	Key dedicated to the management of the SAM itself (personalisation, key management, reloading, authentication).
Work Key	Key used for application-level cryptographic operations.
Event Counter	Numeric counter managed by the SAM, typically used to track the usage of a work key.
Counter Ceiling	Maximum value an event counter is allowed to reach before being considered exhausted.
Unlock Data	Secret value required to unlock a locked SAM. Can be static or dynamic.
Lock File	SAM file controlling whether the SAM is locked or unlocked.
SAM Traceability	Mode in which the signature embeds the signing SAM serial number and the value of the counter associated with the signing key.
Busy Mode	Defensive mode that rate-limits repeated "PSO Verify Signature" attempts after a failure.

3.2. Acronyms

Abbreviation	Expansion
AID	Application IDentifier
APDU	Application Protocol Data Unit
CA	Certification Authority
CNA	Calypso Networks Association
HSM	Hardware Security Module
KIF	Key Identifier
KVC	Key Version Code
PKI	Public Key Infrastructure
PSO	Perform Security Operation (ISO/IEC 7816-8 command class)
SAM	Secure Application Module
SPI	Service Provider Interface
SV	Stored Value
UML	Unified Modelling Language

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

4. Architectural Overview

4.1. Functional positioning

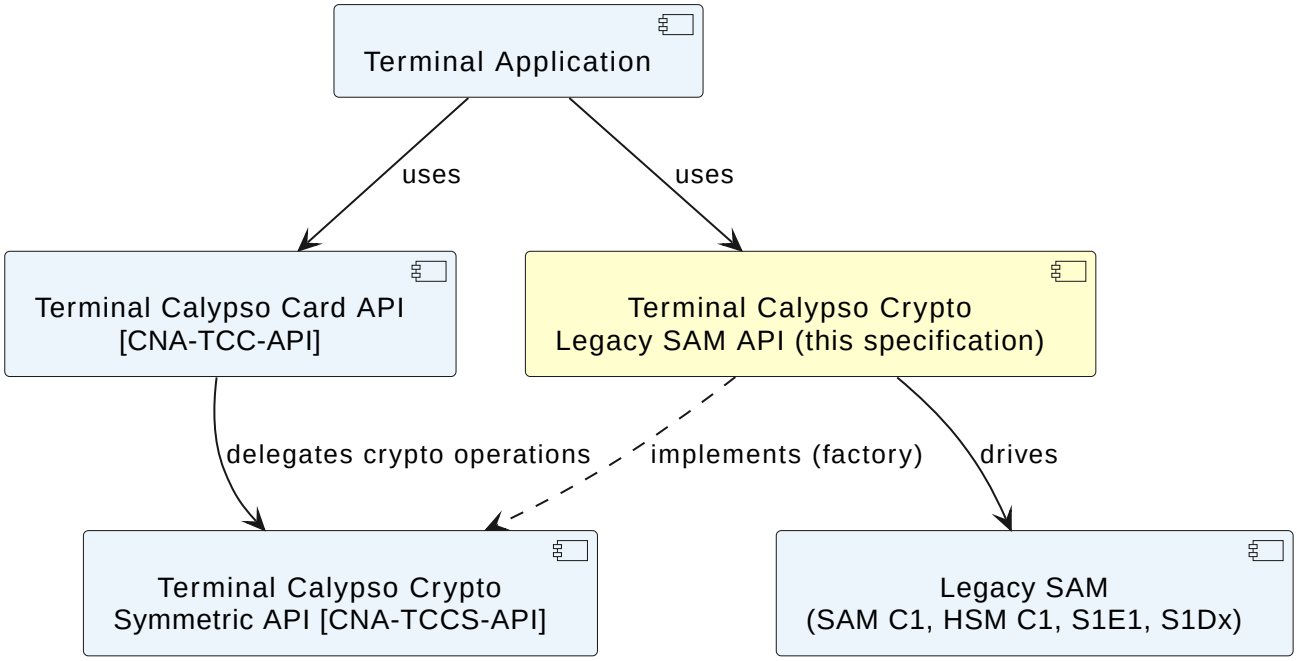


Figure 1. Terminal Calypso Crypto Legacy SAM API — Architecture Overview

4.2. Logical namespaces

Namespace	Role	Summary
`calypso.crypto.legacysam`	Public API	Factory, properties and shared enumerations (system key types, counter increment access, GetData tags).
`calypso.crypto.legacysam.sam`	Public API	SAM data model: `LegacySam`, `KeyParameters`, `LegacySamSelectionExtension`.
`calypso.crypto.legacysam.spi`	SPI	Application-implemented SPIs for unlock data computation and SAM revocation.
`calypso.crypto.legacysam.transaction`	Public API	Transaction managers, signature/key/certificate data carriers, security settings, traceability mode and errors.

4.3. UML class diagram

The following UML class diagram provides a visual representation of the types and relationships defined by this specification:

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

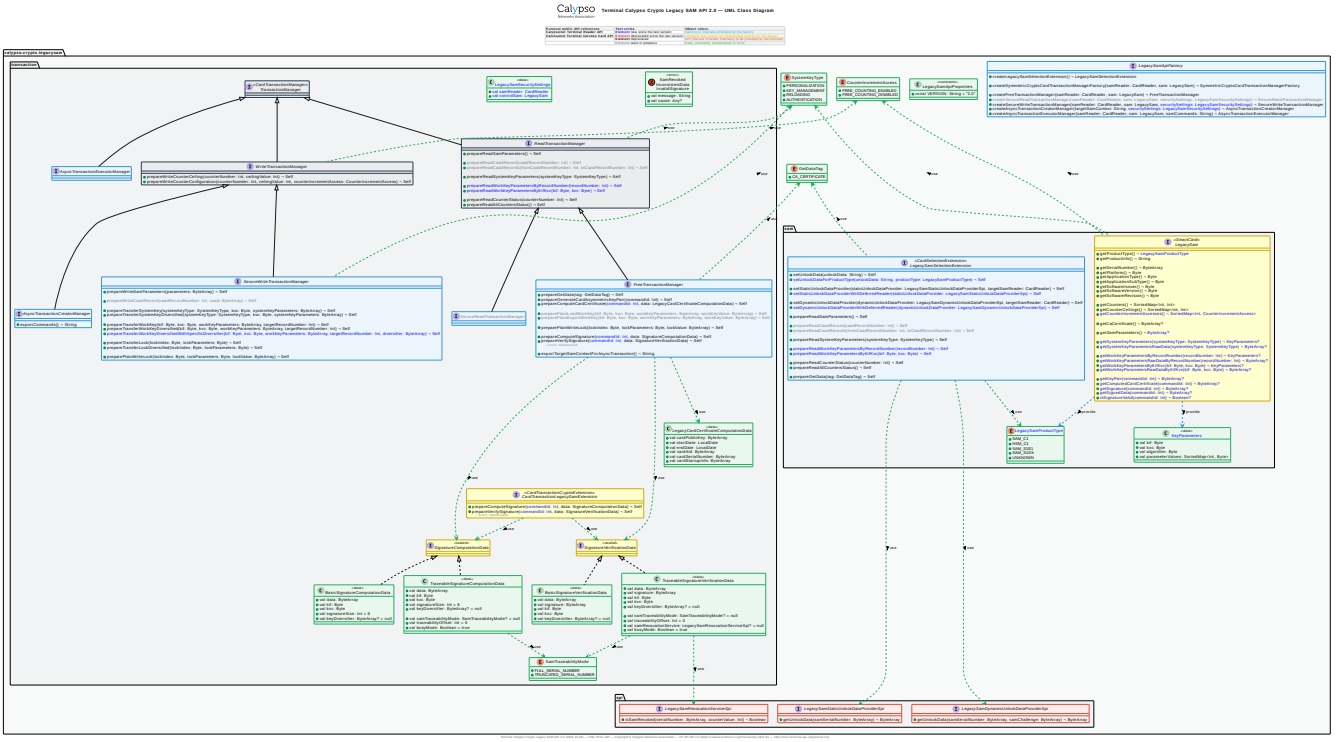


Figure 2. Terminal Calypso Crypto Legacy SAM API — UML Class Diagram

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

5. API Specification

5.1. Constants

5.1.1. Legacy SAM API Properties

Name	`LegacySamApiProperties`
Kind	Constants
Namespace	`calypso.crypto.legacysam`
Since	0.1
Purpose	Exposes the immutable properties of the Terminal Calypso Crypto Legacy SAM API.

Name	Type	Since	Description
`VERSION`	`String`	0.1	String representation of the version of the API implemented by the conforming binding. The value is a dotted decimal of the form `MAJOR.MINOR`; it is `"2.0"` for this revision of the specification.

5.2. Data

5.2.1. Basic Signature Computation Data

Name	`BasicSignatureComputationData`
Kind	Data
Namespace	`calypso.crypto.legacysam.transaction`
Implements	`SignatureComputationData`
Since	0.1
Purpose	Criteria of a basic signature computation, performed by the **Data Cipher** command.

Name	Type	Default	Since	Description
`data`	`ByteArray`	—	0.1	Data to be signed. Non-empty.
`kif`	`Byte`	—	0.1	KIF of the key used for the signature computation.
`kvc`	`Byte`	—	0.1	KVC of the key used for the signature computation.
`signatureSize`	`Int`	`8`	0.1	Expected signature size in bytes, in range `[1..8]`. The longer the signature, the more secure it is.
`keyDiversifier`	`ByteArray?`	`null`	0.1	Explicit 1-to-8-byte key diversifier. `null` leaves the default in force, which is the full serial number of the target card or SAM.

5.2.2. Basic Signature Verification Data

Name	`BasicSignatureVerificationData`
Kind	Data
Namespace	`calypso.crypto.legacysam.transaction`
Implements	`SignatureVerificationData`
Since	0.1

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Purpose	Criteria of a basic signature verification, performed by the **Data Cipher** command.

Name	Type	Default	Since	Description
`data`	`ByteArray`	—	0.1	Signed data. Non-empty.
`signature`	`ByteArray`	—	0.1	Signature associated with the data. Non-empty.
`kif`	`Byte`	—	0.1	KIF of the key used for the signature verification.
`kvc`	`Byte`	—	0.1	KVC of the key used for the signature verification.
`keyDiversifier`	`ByteArray?`	`null`	0.1	Explicit 1-to-8-byte key diversifier. `null` leaves the default in force, which is the full serial number of the target card or SAM.

5.2.3. Key Parameters

Name	`KeyParameters`
Kind	Data
Namespace	`calypso.crypto.legacysam.sam`
Since	0.7
Purpose	Carries the parameters of a key managed by the SAM, system or work key.

Name	Type	Default	Since	Description
`kif`	`Byte`	—	0.7	Key identifier (KIF).
`kvc`	`Byte`	—	0.7	Key version (KVC).
`algorithm`	`Byte`	—	0.7	Key algorithm identifier byte.
`parameterValues`	`SortedMap<Int, Byte>`	—	0.7	Value of each parameter, indexed by parameter number. Contains one entry for every parameter number in range `[1..10]`, that is PAR1 to PAR10.

5.2.4. Legacy Card Certificate Computation Data

Name	`LegacyCardCertificateComputationData`
Kind	Data
Namespace	`calypso.crypto.legacysam.transaction`
Since	0.5
Purpose	Criteria of a card certificate computation, performed by the **PSO Compute Certificate** command.

Name	Type	Default	Since	Description
`cardPublicKey`	`ByteArray`	—	0.5	64-byte public key of the target card, on the secp256r1 curve.
`startDate`	`LocalDate`	—	0.5	Date from which the certificate is valid.
`endDate`	`LocalDate`	—	0.5	Date after which the certificate is no longer valid.
`cardAid`	`ByteArray`	—	0.5	AID of the autonomous PKI application of the target card, 5 to 16 bytes, not all zero.
`cardSerialNumber`	`ByteArray`	—	0.5	8-byte serial number of the target card.
`cardStartupInfo`	`ByteArray`	—	0.5	7-byte startup info to embed in the certificate.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

5.2.5. Legacy SAM Security Settings

Name	`LegacySamSecuritySettings`
Kind	Data
Namespace	`calypso.crypto.legacysam.transaction`
Since	0.2
Purpose	Security settings of a legacy-SAM transaction secured by a control SAM, built directly by the application.

Name	Type	Default	Since	Description
`samReader`	`CardReader` (CNA-TR-API)	—	0.2	Reader through which the control SAM is accessed.
`controlSam`	`LegacySam`	—	0.2	Control SAM used to secure the transaction. Its product type MUST NOT be `UNKNOWN`.

5.2.6. Traceable Signature Computation Data

Name	`TraceableSignatureComputationData`
Kind	Data
Namespace	`calypso.crypto.legacysam.transaction`
Implements	`SignatureComputationData`
Since	0.1
Purpose	Criteria of a traceable signature computation, performed by the **PSO Compute Signature** command.

Name	Type	Default	Since	Description
`data`	`ByteArray`	—	0.1	Data to be signed. Non-empty.
`kif`	`Byte`	—	0.1	KIF of the key used for the signature computation.
`kvc`	`Byte`	—	0.1	KVC of the key used for the signature computation.
`signatureSize`	`Int`	`8`	0.1	Expected signature size in bytes, in range `[1..8]`. The longer the signature, the more secure it is.
`keyDiversifier`	`ByteArray?`	`null`	0.1	Explicit 1-to-8-byte key diversifier. `null` leaves the default in force, which is the full serial number of the target card or SAM.
`samTraceabilityMode`	`SamTraceabilityMode?`	`null`	0.1	Traceability mode to apply. `null` disables **SAM traceability**, which is the default; when set, the SAM inserts its traceability information into the signed data.
`traceabilityOffset`	`Int`	`0`	0.1	Bit offset at which the traceability information is inserted in the data to sign. Relevant only when `samTraceabilityMode` is set.
`busyMode`	`Boolean`	`true`	0.1	When true, the command is issued in **Busy** mode: after a failure the SAM rejects any further command of the same kind for a few seconds. Keeping it enabled is RECOMMENDED for security-sensitive flows.

5.2.7. Traceable Signature Verification Data

Name	`TraceableSignatureVerificationData`
Kind	Data

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Namespace	`calypso.crypto.legacysam.transaction`
Implements	`SignatureVerificationData`
Since	0.1
Purpose	Criteria of a traceable signature verification, performed by the **PSO Verify Signature** command.

Name	Type	Default	Since	Description
`data`	`ByteArray`	—	0.1	Signed data. Non-empty.
`signature`	`ByteArray`	—	0.1	Signature associated with the data. Non-empty.
`kif`	`Byte`	—	0.1	KIF of the key used for the signature verification.
`kvc`	`Byte`	—	0.1	KVC of the key used for the signature verification.
`keyDiversifier`	`ByteArray?`	`null`	0.1	Explicit 1-to-8-byte key diversifier. `null` leaves the default in force, which is the full serial number of the target card or SAM.
`samTraceabilityMode`	`SamTraceabilityMode?`	`null`	0.1	Traceability mode used when the signature was computed. `null` means the signature was not computed in **SAM traceability** mode, which is the default.
`traceabilityOffset`	`Int`	`0`	0.1	Bit offset at which the traceability information is inserted in the data to sign. Relevant only when `samTraceabilityMode` is set.
`samRevocationService`	`LegacySamRevocationServiceSpi?`	`null`	0.1	Service used to check the revocation status of the signing SAM. `null` skips the check.
`busyMode`	`Boolean`	`true`	0.1	When true, the command is issued in **Busy** mode: after a failure the SAM rejects any further command of the same kind for a few seconds. Keeping it enabled is RECOMMENDED for security-sensitive flows.

5.3. API Interfaces

5.3.1. Async Transaction Creator Manager

Name	`AsyncTransactionCreatorManager`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Extends	`WriteTransactionManager`
Since	0.2
Purpose	Prepares write commands using a control SAM and exports them as a serialised stream for later replay by an `AsyncTransactionExecutorManager`. An instance is obtained via `LegacySamApiFactory.createAsyncTransactionCreatorManager`.
See also	`AsyncTransactionExecutorManager`

Export Commands

Signature	`exportCommands() → String`
Since	0.2
Description	Returns a non-empty `String` carrying the prepared commands. These commands can later be imported and processed by an `AsyncTransactionExecutorManager`.
Returns	A non-empty `String`.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Pre-conditions	None.
Errors	None.

5.3.2. Async Transaction Executor Manager

Name	`AsyncTransactionExecutorManager`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Extends	`TransactionManager`
Since	0.2
Purpose	Executes a command stream prepared by an `AsyncTransactionCreatorManager`. Declares no additional operation. An instance is obtained via `LegacySamApiFactory.createAsyncTransactionExecutorManager`.
See also	`AsyncTransactionCreatorManager`

5.3.3. Card Transaction Legacy SAM Extension

Name	`CardTransactionLegacySamExtension`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Extends	`CardTransactionCryptoExtension` (CNA-TCC-API)
Since	0.3
Purpose	Card-transaction crypto extension enriching the CNA-TCC-API command set with SAM-backed signature computation and verification. Instances are obtained through `SecureRegularModeTransactionManager.getCryptoExtension` or `SecureExtendedModeTransactionManager.getCryptoExtension`.

Prepare Compute Signature

Signature	`prepareComputeSignature(`
	`  commandId: Int,`
	`  data: SignatureComputationData`
	`) → Self`

Since	0.3
Description	Schedules the execution of a **Data Cipher** or **PSO Compute Signature** command in the context of a Calypso card transaction. Once processed, the signature is available in the `LegacySam` through `getSignature` called with `commandId`, and, for a traceable computation, the signed data through `getSignedData`. The signature serves many purposes, for example: · **To sign data recorded in a contactless card or ticket.** To speed up processing, it is RECOMMENDED to use a constant signing key (not diversified before ciphering) and to insert the serial number of the card or ticket at the beginning of the data to sign. · **To sign data reported from a terminal to a central system.** The terminal SAM carries a signing work key diversified with its own serial number, which guarantees that the data was indeed signed by this SAM. The central system SAM uses the master signing key, diversified before signing with the diversifier set previously by the **Select Diversifier** command.
Parameters	`commandId` — the application-supplied identifier of this command. `data` (`SignatureComputationData`) — the computation criteria.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — the input data is consistent.
Errors	None.

See also	`SignatureComputationData`
	`BasicSignatureComputationData`
	`TraceableSignatureComputationData`

Prepare Verify Signature

Signature	`prepareVerifySignature(`
	`  commandId: Int,`
	`  data: SignatureVerificationData`
	`) → Self`

Since	0.3
Description	Schedules the execution of a **Data Cipher** or **PSO Verify Signature** command in the context of a Calypso card transaction. Once processed, the outcome is available in the `LegacySam` through `isSignatureValid` called with `commandId`.
Parameters	`commandId` — the application-supplied identifier of this command. `data` (`SignatureVerificationData`) — the verification criteria.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — the input data is consistent.
Errors	`SamRevoked` — if the signature was computed in **SAM traceability** mode, the revocation check was requested and the SAM is revoked (traceable signatures only).

See also	`SignatureVerificationData`
	`BasicSignatureVerificationData`
	`TraceableSignatureVerificationData`

5.3.4. Free Transaction Manager

Name	`FreeTransactionManager`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Extends	`ReadTransactionManager`
Since	0.1
Purpose	Transaction manager used when no control SAM is involved. An instance is obtained via `LegacySamApiFactory.createFreeTransactionManager`.

Export Target SAM Context For Async Transaction

Signature	`exportTargetSamContextForAsyncTransaction() → String`
Since	0.2
Description	Executes the commands required to obtain the security context of the target SAM. The returned `String` is passed to `LegacySamApiFactory.createAsyncTransactionCreatorManager`.
Returns	A non-empty `String`.
Pre-conditions	None.
Errors	None.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Prepare Compute Card Certificate

Signature	`prepareComputeCardCertificate(`
	`  commandId: Int,`
	`  data: LegacyCardCertificateComputationData`
	`) → Self`

Since	0.5
Description	Schedules the execution of a **PSO Compute Certificate** command. Once processed, the certificate is available in the `LegacySam` through `getComputedCardCertificate` called with `commandId`.
Parameters	`commandId` — the application-supplied identifier of this command. `data` (`LegacyCardCertificateComputationData`) — the computation criteria.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.
See also	`LegacyCardCertificateComputationData` `LegacySam.getComputedCardCertificate`

Prepare Compute Signature

Signature	`prepareComputeSignature(`
	`  commandId: Int,`
	`  data: SignatureComputationData`
	`) → Self`

Since	0.1
Description	Schedules the execution of a **Data Cipher** or **PSO Compute Signature** command. Once processed, the signature is available in the `LegacySam` through `getSignature` called with `commandId`, and, for a traceable computation, the signed data through `getSignedData`. The signature serves many purposes, for example: · **To sign data recorded in a contactless card or ticket.** To speed up processing, it is RECOMMENDED to use a constant signing key (not diversified before ciphering) and to insert the serial number of the card or ticket at the beginning of the data to sign. · **To sign data reported from a terminal to a central system.** The terminal SAM carries a signing work key diversified with its own serial number, which guarantees that the data was indeed signed by this SAM. The central system SAM uses the master signing key, diversified before signing with the diversifier set previously by the **Select Diversifier** command.
Parameters	`commandId` — the application-supplied identifier of this command. `data` (`SignatureComputationData`) — the computation criteria.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — the input data is consistent.
Errors	None.

See also	`SignatureComputationData`
	`BasicSignatureComputationData`
	`TraceableSignatureComputationData`
	`LegacySam.getSignature`

Prepare Generate Card Asymmetric Key Pair

Signature	`prepareGenerateCardAsymmetricKeyPair(commandId: Int) → Self`
Since	0.5

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Description	Schedules the execution of a **Card Generate Asymmetric Key Pair** command. Once processed, the key pair is available in the `LegacySam` through `getKeyPair` called with `commandId`.
Parameters	`commandId` — the application-supplied identifier of this command.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.
See also	`LegacySam.getKeyPair`

Prepare Get Data

Signature	`prepareGetData(tag: GetDataTag) → Self`
Since	0.5
Description	Schedules the execution of a **Get Data** command for the supplied tag. Once processed, the data is accessible through the dedicated getters of the `LegacySam`, such as `getCaCertificate`.
Parameters	`tag` (`GetDataTag`) — the tag to retrieve the data for.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

Prepare Plain Write Lock

Signature	`preparePlainWriteLock(`
	`  lockIndex: Byte,`
	`  lockParameters: Byte,`
	`  lockValue: ByteArray`
	`) → Self`

Since	0.7
Description	Schedules the execution of a **Write Key** command to set the lock file of the SAM. The lock value is transferred in plain text.
Parameters	`lockIndex` — the index of the lock file. `lockParameters` — the lock permissions parameters. `lockValue` — a 16-byte `ByteArray` representing the lock's value.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — `lockValue` is within range.
Errors	None.

Prepare Verify Signature

Signature	`prepareVerifySignature(`
	`  commandId: Int,`
	`  data: SignatureVerificationData`
	`) → Self`

Since	0.1
Description	Schedules the execution of a **Data Cipher** or **PSO Verify Signature** command. Once processed, the outcome is available in the `LegacySam` through `isSignatureValid` called with `commandId`.
Parameters	`commandId` — the application-supplied identifier of this command. `data` (`SignatureVerificationData`) — the verification criteria.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — the input data is consistent.
Errors	`SamRevoked` — if the signature was computed in **SAM traceability** mode, the revocation check was requested and the SAM is revoked (traceable signatures only).

See also	`SignatureVerificationData`
	`BasicSignatureVerificationData`
	`TraceableSignatureVerificationData`
	`LegacySam.isSignatureValid`

5.3.5. Legacy SAM

Name	`LegacySam`
Kind	Interface
Namespace	`calypso.crypto.legacysam.sam`
Extends	`SmartCard` (CNA-TR-API)
Since	0.1
Purpose	Dynamic view of a legacy SAM, updated from selection to the end of the transaction.

A `LegacySam` instance is obtained by casting the CNA-TR-API `SmartCard` produced by the selection scenario when the selection extension was a `LegacySamSelectionExtension`.

As a **dynamic view** of the SAM image, a `LegacySam` is updated after every successful command processed by a **synchronous** transaction manager; **asynchronous** write managers do not update the local `LegacySam`.

Get Application Sub Type

Signature	`getApplicationSubType() → Byte`
Since	0.1
Description	Returns the application subtype byte.
Returns	A `Byte` value.
Pre-conditions	None.
Errors	None.

Get Application Type

Signature	`getApplicationType() → Byte`
Since	0.1
Description	Returns the application type byte.
Returns	A `Byte` value.
Pre-conditions	None.
Errors	None.

Get CA Certificate

Signature	`getCaCertificate() → ByteArray?`
Since	0.5

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Description	Returns the CA certificate retrieved from the SAM (384 bytes); `null` if not available.
Returns	A non-empty `ByteArray`, or `null` if not available.
Pre-conditions	None.
Errors	None.
See also	`LegacySamSelectionExtension.prepareGetData` `FreeTransactionManager.prepareGetData`

Get Computed Card Certificate

Signature	`getComputedCardCertificate(commandId: Int) → ByteArray?`
Since	2.0
Description	316-byte card certificate produced by the certificate computation prepared with this identifier. When several such commands share the same identifier, the value returned is that of the most recently processed one.
Parameters	`commandId` — the identifier attached to the command when it was prepared.
Returns	A non-empty `ByteArray`. `null` if no certificate computation carrying this identifier has been processed.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.
See also	`FreeTransactionManager.prepareComputeCardCertificate`

Get Counter Ceilings

Signature	`getCounterCeilings() → SortedMap<Int, Int>`
Since	0.1
Description	Returns the known ceiling values, keyed by ceiling number.
Returns	A `SortedMap<Int, Int>`; empty if no ceiling is known.
Pre-conditions	None.
Errors	None.
See also	`LegacySamSelectionExtension.prepareReadAllCountersStatus`

Get Counter Increment Accesses

Signature	`getCounterIncrementAccesses() → SortedMap<Int, CounterIncrementAccess>`
Since	2.0
Description	Returns the known increment-access modes, keyed by counter number.
Returns	A `SortedMap<Int, CounterIncrementAccess>`; empty if no mode is known.
Pre-conditions	None.
Errors	None.
See also	`LegacySamSelectionExtension.prepareReadCounterStatus`

Get Counters

Signature	`getCounters() → SortedMap<Int, Int>`
Since	0.1

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Description	Returns the values of the known counters, keyed by counter number.
Returns	A `SortedMap<Int, Int>`; empty if no counter is known.
Pre-conditions	None.
Errors	None.
See also	`LegacySamSelectionExtension.prepareReadAllCountersStatus`

Get Key Pair

Signature	`getKeyPair(commandId: Int) → ByteArray?`
Since	2.0
Description	96-byte key pair produced by the card asymmetric key pair generation prepared with this identifier. When several such commands share the same identifier, the value returned is that of the most recently processed one.
Parameters	`commandId` — the identifier attached to the command when it was prepared.
Returns	A non-empty `ByteArray`. `null` if no key pair generation carrying this identifier has been processed.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.
See also	`FreeTransactionManager.prepareGenerateCardAsymmetricKeyPair`

Get Platform

Signature	`getPlatform() → Byte`
Since	0.1
Description	Returns the platform identifier byte.
Returns	A `Byte` value.
Pre-conditions	None.
Errors	None.

Get Product Info

Signature	`getProductInfo() → String`
Since	0.1
Description	Returns a textual description of the SAM.
Returns	A non-empty `String`.
Pre-conditions	None.
Errors	None.

Get Product Type

Signature	`getProductType() → LegacySamProductType`
Since	0.1
Description	Returns the SAM product type. Possible values: `SAM_C1`, `HSM_C1`, `SAM_S1E1`, `SAM_S1DX`, `UNKNOWN`.
Returns	A `LegacySamProductType`.
Pre-conditions	None.
Errors	None.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Get SAM Parameters

Signature	`getSamParameters() → ByteArray?`
Since	0.7
Description	Returns the raw data of the SAM parameters as a 29-byte array; `null` if not available.
Returns	A non-empty `ByteArray`, or `null` if not available.
Pre-conditions	None.
Errors	None.
See also	`LegacySamSelectionExtension.prepareReadSamParameters`

Get Serial Number

Signature	`getSerialNumber() → ByteArray`
Since	0.1
Description	Returns the SAM serial number.
Returns	A non-empty `ByteArray`.
Pre-conditions	None.
Errors	None.

Get Signature

Signature	`getSignature(commandId: Int) → ByteArray?`
Since	2.0
Description	Signature computed by the signature computation prepared with this identifier. When several such commands share the same identifier, the value returned is that of the most recently processed one.
Parameters	`commandId` — the identifier attached to the command when it was prepared.
Returns	A non-empty `ByteArray` of 1 to 8 bytes. `null` if no signature computation carrying this identifier has been processed.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.
See also	`FreeTransactionManager.prepareComputeSignature`

Get Signed Data

Signature	`getSignedData(commandId: Int) → ByteArray?`
Since	2.0
Description	Data actually signed by the traceable signature computation prepared with this identifier. When **SAM traceability** mode was enabled, it embeds the SAM traceability information. When several such commands share the same identifier, the value returned is that of the most recently processed one.
Parameters	`commandId` — the identifier attached to the command when it was prepared.
Returns	A non-empty `ByteArray`. `null` if no traceable signature computation carrying this identifier has been processed.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.
See also	`FreeTransactionManager.prepareComputeSignature`

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Get Software Issuer

Signature	`getSoftwareIssuer() → Byte`
Since	0.1
Description	Returns the software issuer identifier.
Returns	A `Byte` value.
Pre-conditions	None.
Errors	None.

Get Software Revision

Signature	`getSoftwareRevision() → Byte`
Since	0.1
Description	Returns the software revision number.
Returns	A `Byte` value.
Pre-conditions	None.
Errors	None.

Get Software Version

Signature	`getSoftwareVersion() → Byte`
Since	0.1
Description	Returns the software version number.
Returns	A `Byte` value.
Pre-conditions	None.
Errors	None.

Get System Key Parameters

Signature	`getSystemKeyParameters(systemKeyType: SystemKeyType) → KeyParameters?`
Since	0.2
Description	Returns the parameters of the supplied system key; `null` if not available. Each call returns a snapshot decoded from the value returned by `getSystemKeyParametersRawData` at the time of the call.
Parameters	`systemKeyType` (`SystemKeyType`) — the system key whose parameters are requested.
Returns	A `KeyParameters`, or `null` if not available.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.
See also	`getSystemKeyParametersRawData` `LegacySamSelectionExtension.prepareReadSystemKeyParameters`

Get System Key Parameters Raw Data

Signature	`getSystemKeyParametersRawData(systemKeyType: SystemKeyType) → ByteArray?`
Since	2.0

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Description	Returns the raw parameters of the supplied system key: 13 bytes carrying KIF, KVC, algorithm and PAR1..PAR10; `null` if not available. The `KeyParameters` returned by `getSystemKeyParameters` for the same key at the same time is the decoding of this value; both are `null` together.
Parameters	`systemKeyType` (`SystemKeyType`) — the system key whose parameters are requested.
Returns	The 13-byte raw key parameters, or `null` if not available.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.
See also	`getSystemKeyParameters`

Get Work Key Parameters By KIF KVC

Signature	`getWorkKeyParametersByKifKvc(`
	`  kif: Byte,`
	`  kvc: Byte`
	`) → KeyParameters?`

Since	0.7
Description	Returns the parameters of the work key referenced by its KIF/KVC pair; `null` if not available. Each call returns a snapshot decoded from the value returned by `getWorkKeyParametersRawDataByKifKvc` at the time of the call.
Parameters	`kif` — the key KIF. `kvc` — the key KVC.
Returns	A `KeyParameters`, or `null` if not available.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.
See also	`getWorkKeyParametersRawDataByKifKvc` `LegacySamSelectionExtension.prepareReadWorkKeyParametersByKifKvc`

Get Work Key Parameters By Record Number

Signature	`getWorkKeyParametersByRecordNumber(recordNumber: Int) → KeyParameters?`
Since	0.7
Description	Returns the parameters of the work key at the supplied record number (`[1..126]`); `null` if not available. Each call returns a snapshot decoded from the value returned by `getWorkKeyParametersRawDataByRecordNumber` at the time of the call.
Parameters	`recordNumber` — the key record number (in range [1..126]).
Returns	A `KeyParameters`, or `null` if not available.
Pre-conditions	**Argument** — every parameter is non-`null`. **Range** — the record number is within range.
Errors	None.
See also	`getWorkKeyParametersRawDataByRecordNumber` `LegacySamSelectionExtension.prepareReadWorkKeyParametersByRecordNumber`

Get Work Key Parameters Raw Data By KIF KVC

Signature	`getWorkKeyParametersRawDataByKifKvc(`
	`  kif: Byte,`
	`  kvc: Byte`
	`) → ByteArray?`

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Since	2.0
Description	Returns the raw parameters of the work key referenced by its KIF/KVC pair: 13 bytes carrying KIF, KVC, algorithm and PAR1..PAR10; `null` if not available. The `KeyParameters` returned by `getWorkKeyParametersByKifKvc` for the same key at the same time is the decoding of this value; both are `null` together.
Parameters	`kif` — the key KIF. `kvc` — the key KVC.
Returns	The 13-byte raw key parameters, or `null` if not available.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.
See also	`getWorkKeyParametersByKifKvc`

Get Work Key Parameters Raw Data By Record Number

Signature	`getWorkKeyParametersRawDataByRecordNumber(recordNumber: Int) → ByteArray?`
Since	2.0
Description	Returns the raw parameters of the work key at the supplied record number (`[1..126]`): 13 bytes carrying KIF, KVC, algorithm and PAR1..PAR10; `null` if not available. The `KeyParameters` returned by `getWorkKeyParametersByRecordNumber` for the same key at the same time is the decoding of this value; both are `null` together.
Parameters	`recordNumber` — the key record number (in range [1..126]).
Returns	The 13-byte raw key parameters, or `null` if not available.
Pre-conditions	**Argument** — every parameter is non-`null`. **Range** — the record number is within range.
Errors	None.
See also	`getWorkKeyParametersByRecordNumber`

Is Signature Valid

Signature	`isSignatureValid(commandId: Int) → Boolean?`
Since	2.0
Description	Outcome of the signature verification prepared with this identifier. When several such commands share the same identifier, the value returned is that of the most recently processed one.
Parameters	`commandId` — the identifier attached to the command when it was prepared.
Returns	A `Boolean` value. `null` if no signature verification carrying this identifier has been processed.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.
See also	`FreeTransactionManager.prepareVerifySignature`

5.3.6. Legacy SAM API Factory

Name	`LegacySamApiFactory`
Kind	Interface
Namespace	`calypso.crypto.legacysam`
Since	0.3
Purpose	Factory used by the application to obtain instances of every public type provided by the API.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Create Async Transaction Creator Manager

Signature	`createAsyncTransactionCreatorManager(`
	`  targetSamContext: String,`
	`  securitySettings: LegacySamSecuritySettings`
	`) → AsyncTransactionCreatorManager`

Since	0.3
Description	Returns a new `AsyncTransactionCreatorManager`. The target SAM context MUST have been produced by `FreeTransactionManager.exportTargetSamContextForAsyncTransaction`.
Parameters	`targetSamContext` — the target SAM context. `securitySettings` (`LegacySamSecuritySettings`) — the security settings.
Returns	An `AsyncTransactionCreatorManager`.
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — every argument is valid.
Errors	None.

Create Async Transaction Executor Manager

Signature	`createAsyncTransactionExecutorManager(`
	`  samReader: CardReader,`
	`  sam: LegacySam,`
	`  samCommands: String`
	`) → AsyncTransactionExecutorManager`

Since	0.3
Description	Returns a new `AsyncTransactionExecutorManager`. The `samCommands` MUST have been produced by an `AsyncTransactionCreatorManager`.
Parameters	`samReader` (`CardReader` (CNA-TR-API)) — the reader to use to communicate with the SAM. `sam` (`LegacySam`) — the SAM image. `samCommands` — a `String` containing the prepared commands.
Returns	An `AsyncTransactionExecutorManager`.
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — every argument is valid.
Errors	None.

Create Free Transaction Manager

Signature	`createFreeTransactionManager(`
	`  samReader: CardReader,`
	`  sam: LegacySam`
	`) → FreeTransactionManager`

Since	0.3
Description	Returns a new `FreeTransactionManager` bound to the supplied SAM and reader.
Parameters	`samReader` (`CardReader` (CNA-TR-API)) — the reader to use to communicate with the SAM. `sam` (`LegacySam`) — the SAM image.
Returns	A `FreeTransactionManager`.
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — every argument is valid.
Errors	None.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Create Legacy SAM Selection Extension

Signature	`createLegacySamSelectionExtension() → LegacySamSelectionExtension`
Since	0.3
Description	Creates a `LegacySamSelectionExtension` that enriches a CNA-TR-API selection scenario with SAM-specific commands (Unlock, Read Parameters, etc.).
Returns	A `LegacySamSelectionExtension`.
Pre-conditions	None.
Errors	None.

Create Secure Write Transaction Manager

Signature	`createSecureWriteTransactionManager(`
	`  samReader: CardReader,`
	`  sam: LegacySam,`
	`  securitySettings: LegacySamSecuritySettings`
	`) → SecureWriteTransactionManager`

Since	0.7
Description	Returns a new `SecureWriteTransactionManager` bound to the supplied SAM, reader and security settings.
Parameters	`samReader` (`CardReader` (CNA-TR-API)) — the reader to use to communicate with the SAM. `sam` (`LegacySam`) — the SAM image. `securitySettings` (`LegacySamSecuritySettings`) — the security settings.
Returns	A `SecureWriteTransactionManager`.
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — every argument is valid.
Errors	None.

Create Symmetric Crypto Card Transaction Manager Factory

Signature	`createSymmetricCryptoCardTransactionManagerFactory(`
	`  samReader: CardReader,`
	`  sam: LegacySam`
	`) → SymmetricCryptoCardTransactionManagerFactory`

Since	0.3
Description	Returns a new factory of CNA-TCCS-API `SymmetricCryptoCardTransactionManager` to be used to secure a Calypso card transaction with the supplied legacy SAM.
Parameters	`samReader` (`CardReader` (CNA-TR-API)) — the reader to use to communicate with the SAM. `sam` (`LegacySam`) — the associated control SAM to be used with the card transaction.
Returns	A `SymmetricCryptoCardTransactionManagerFactory` (CNA-TCC-API).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — every argument is valid.
Errors	None.

5.3.7. Legacy SAM Selection Extension

Name	`LegacySamSelectionExtension`
Kind	Interface
Namespace	`calypso.crypto.legacysam.sam`

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Extends	`CardSelectionExtension` (CNA-TR-API)
Since	0.3
Purpose	Enriches the CNA-TR-API selection scenario with SAM-specific commands (Unlock, Read Parameters, etc.). An instance is obtained via `LegacySamApiFactory.createLegacySamSelectionExtension`.

If the SAM is **locked**, three mutually exclusive unlocking strategies are offered:

1. The application supplies directly the 16-byte unlock value expected by the SAM (static mode)—see `setUnlockData`.
2. The application supplies a `LegacySamStaticUnlockDataProviderSpi` to compute the unlock value (possibly diversified with the SAM serial number).
3. The application supplies a `LegacySamDynamicUnlockDataProviderSpi` to obtain the 8-byte value expected by the SAM in dynamic mode (computed by an origin SAM).

When the unlocking data is supplied by a provider, a CNA-TR-API `CardReader` is required for additional exchanges. The reader MAY be provided either at selection-extension creation time, or later when using `setStaticUnlockDataProviderWithDeferredReader` or `setDynamicUnlockDataProviderWithDeferredReader` (e.g. dynamic SAM reader allocation).

Prepare Get Data

Signature	`prepareGetData(tag: GetDataTag) → Self`
Since	0.6
Description	Schedules the execution of a **Get Data** command for the supplied tag. The result is accessible via a dedicated getter (e.g. `LegacySam.getCaCertificate`).
Parameters	`tag` (`GetDataTag`)—the tag to retrieve the data for.
Returns	The current instance (`Self`).
Pre-conditions	**Argument**—every parameter is non-`null`.
Errors	None.

Prepare Read All Counters Status

Signature	`prepareReadAllCountersStatus() → Self`
Since	0.3
Description	Schedules the reading of the status of every counter.
Returns	The current instance (`Self`).
Pre-conditions	None.
Errors	None.

Prepare Read Counter Status

Signature	`prepareReadCounterStatus(counterNumber: Int) → Self`
Since	0.3
Description	Schedules the execution of **Read Event Counter** and **Read Ceiling** commands to read the status of the supplied counter (`[0..26]`). The actual number of commands sent to the SAM will be reduced by reading the whole record at once.
Parameters	`counterNumber`—the number of the counter whose status is to be read (in range `[0..26]`).
Returns	The current instance (`Self`).

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Pre-conditions	**Argument**—every parameter is non-`null`. **Argument**—the counter number is within range.
Errors	None.

Prepare Read SAM Parameters

Signature	`prepareReadSamParameters() → Self`
Since	0.7
Description	Schedules the execution of a **Read Parameters** command. The result is available via `LegacySam.getSamParameters`.
Returns	The current instance (`Self`).
Pre-conditions	None.
Errors	None.

Prepare Read System Key Parameters

Signature	`prepareReadSystemKeyParameters(`
	`  systemKeyType: SystemKeyType`
	`) → Self`

Since	0.3
Description	Schedules the execution of a **Read Key Parameters** command for the supplied system key. The result is available via `LegacySam.getSystemKeyParameters`.
Parameters	`systemKeyType` (`SystemKeyType`)—the type of system key.
Returns	The current instance (`Self`).
Pre-conditions	**Argument**—every parameter is non-`null`.
Errors	None.

Prepare Read Work Key Parameters By KIF KVC

Signature	`prepareReadWorkKeyParametersByKifKvc(`
	`  kif: Byte,`
	`  kvc: Byte`
	`) → Self`

Since	0.7
Description	Schedules the execution of a **Read Key Parameters** command for the work key referenced by its KIF/KVC pair. Once processed, the result is accessible through `LegacySam.getWorkKeyParametersByKifKvc`.
Parameters	`kif`—the key KIF. `kvc`—the key KVC.
Returns	The current instance (`Self`).
Pre-conditions	**Argument**—every parameter is non-`null`.
Errors	None.

Prepare Read Work Key Parameters By Record Number

Signature	`prepareReadWorkKeyParametersByRecordNumber(`
	`  recordNumber: Int`
	`) → Self`

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Since	0.7
Description	Schedules the execution of a **Read Key Parameters** command for the work key at the supplied record number (`[1..126]`). Once processed, the result is accessible through `LegacySam.getWorkKeyParametersByRecordNumber`.
Parameters	`recordNumber` — the key record number (in range `[1..126]`).
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — the record number is within range.
Errors	None.

Set Dynamic Unlock Data Provider

Signature	`setDynamicUnlockDataProvider(`
	`  dynamicUnlockDataProvider: LegacySamDynamicUnlockDataProviderSpi,`
	`  targetSamReader: CardReader`
	`) → Self`

Since	0.4
Description	Sets the dynamic-unlock-data provider, with the target SAM reader supplied at creation time. The **Unlock** command is initiated after a successful filtering, followed by a request to the provider.
Parameters	`dynamicUnlockDataProvider` (`LegacySamDynamicUnlockDataProviderSpi`) — an implementation of `LegacySamDynamicUnlockDataProviderSpi`. `targetSamReader` (`CardReader` (CNA-TR-API)) — the card reader used to communicate with the target SAM.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **State** — no unlocking setting has been set.
Errors	None.

Set Dynamic Unlock Data Provider With Deferred Reader

Signature	`setDynamicUnlockDataProviderWithDeferredReader(`
	`  dynamicUnlockDataProvider: LegacySamDynamicUnlockDataProviderSpi`
	`) → Self`

Since	0.4
Description	Sets the dynamic-unlock-data provider. To be used when the card reader needed to communicate with the target SAM is supplied later in the workflow. The **Unlock** command is initiated after a successful filtering, followed by a request to the provider.
Parameters	`dynamicUnlockDataProvider` (`LegacySamDynamicUnlockDataProviderSpi`) — an implementation of `LegacySamDynamicUnlockDataProviderSpi`.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **State** — no unlocking setting has been set.
Errors	None.

Set Static Unlock Data Provider

Signature	`setStaticUnlockDataProvider(`
	`  staticUnlockDataProvider: LegacySamStaticUnlockDataProviderSpi,`
	`  targetSamReader: CardReader`
	`) → Self`

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Since	0.4
Description	Sets the static-unlock-data provider, with the target SAM reader supplied at creation time. The **Unlock** command is initiated after a successful filtering, followed by a request to the provider.
Parameters	`staticUnlockDataProvider` (`LegacySamStaticUnlockDataProviderSpi`) — an implementation of `LegacySamStaticUnlockDataProviderSpi`. `targetSamReader` (`CardReader` (CNA-TR-API)) — the card reader used to communicate with the target SAM.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **State** — no unlocking setting has been set.
Errors	None.

Set Static Unlock Data Provider With Deferred Reader

Signature	`setStaticUnlockDataProviderWithDeferredReader(`
	`  staticUnlockDataProvider: LegacySamStaticUnlockDataProviderSpi`
	`) → Self`

Since	0.4
Description	Sets the static-unlock-data provider. To be used when the SAM reader is supplied later in the workflow. The **Unlock** command is initiated after a successful filtering, followed by a request to the provider.
Parameters	`staticUnlockDataProvider` (`LegacySamStaticUnlockDataProviderSpi`) — an implementation of `LegacySamStaticUnlockDataProviderSpi`.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **State** — no unlocking setting has been set.
Errors	None.

Set Unlock Data

Signature	`setUnlockData(unlockData: String) → Self`
Since	0.3
Description	Sets the unlock data (8 or 16 bytes, supplied as a 32-character hexadecimal `String`) used to unlock a SAM C1 and schedules an **Unlock** command in the first position of the selection scenario. The **Unlock** command MUST be executed only after a successful filtering.
Parameters	`unlockData` — the unlock data as a 32-character hexadecimal `String`.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — the unlock data is well-formed and within range. **State** — no unlocking setting has been set.
Errors	None.

Set Unlock Data For Product Type

Signature	`setUnlockDataForProductType(`
	`  unlockData: String,`
	`  productType: LegacySamProductType`
	`) → Self`

Since	0.3

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Description	Sets the unlock data (8 or 16 bytes, supplied as a 32-character hexadecimal `String`) used to unlock a SAM C1, with an explicit target product type, and schedules an **Unlock** command in the first position of the selection scenario. The **Unlock** command MUST be executed only after a successful filtering.
Parameters	`unlockData` — the unlock data as a 32-character hexadecimal `String`. `productType` (`LegacySamProductType`) — the targeted product type.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — every argument is well-formed and within range. **State** — no unlocking setting has been set.
Errors	None.

5.3.8. Read Transaction Manager

Name	`ReadTransactionManager`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Extends	`TransactionManager`
Since	0.1
Purpose	Adds **read**-oriented operations to the common transaction manager: SAM parameters, system/work keys and counter status.

The operations on this interface mirror their equivalents on `LegacySamSelectionExtension`, which make the same readings available during the selection.

Prepare Read All Counters Status

Signature	`prepareReadAllCountersStatus() → Self`
Since	0.3
Description	Schedules the reading of the status of every counter.
Returns	The current instance (`Self`).
Pre-conditions	None.
Errors	None.
See also	`LegacySamSelectionExtension.prepareReadAllCountersStatus`

Prepare Read Counter Status

Signature	`prepareReadCounterStatus(counterNumber: Int) → Self`
Since	0.3
Description	Schedules the execution of **Read Event Counter** and **Read Ceiling** commands to read the status of the supplied counter (`[0..26]`). The actual number of commands sent to the SAM will be reduced by reading the whole record at once.
Parameters	`counterNumber` — the number of the counter whose status is to be read (in range `[0..26]`).
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — the counter number is within range.
Errors	None.
See also	`LegacySamSelectionExtension.prepareReadCounterStatus`

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Prepare Read SAM Parameters

Signature	`prepareReadSamParameters() → Self`
Since	0.7
Description	Schedules the execution of a **Read Parameters** command. The result is available via `LegacySam.getSamParameters`.
Returns	The current instance (`Self`).
Pre-conditions	None.
Errors	None.
See also	`LegacySamSelectionExtension.prepareReadSamParameters`

Prepare Read System Key Parameters

Signature	`prepareReadSystemKeyParameters(`
	`  systemKeyType: SystemKeyType`
	`) → Self`

Since	0.3
Description	Schedules the execution of a **Read Key Parameters** command for the supplied system key. The result is available via `LegacySam.getSystemKeyParameters`.
Parameters	`systemKeyType` (`SystemKeyType`) — the type of system key.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.
See also	`LegacySamSelectionExtension.prepareReadSystemKeyParameters`

Prepare Read Work Key Parameters By KIF KVC

Signature	`prepareReadWorkKeyParametersByKifKvc(`
	`  kif: Byte,`
	`  kvc: Byte`
	`) → Self`

Since	0.7
Description	Schedules the execution of a **Read Key Parameters** command for the work key referenced by its KIF/KVC pair. Once processed, the result is accessible through `LegacySam.getWorkKeyParametersByKifKvc`.
Parameters	`kif` — the key KIF. `kvc` — the key KVC.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.
See also	`LegacySamSelectionExtension.prepareReadWorkKeyParametersByKifKvc`

Prepare Read Work Key Parameters By Record Number

Signature	`prepareReadWorkKeyParametersByRecordNumber(`
	`  recordNumber: Int`
	`) → Self`

Since	0.7

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Description	Schedules the execution of a **Read Key Parameters** command for the work key at the supplied record number (`[1..126]`). Once processed, the result is accessible through `LegacySam.getWorkKeyParametersByRecordNumber`.
Parameters	`recordNumber` — the key record number (in range `[1..126]`).
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — the record number is within range.
Errors	None.
See also	`LegacySamSelectionExtension.prepareReadWorkKeyParametersByRecordNumber`

5.3.9. Secure Write Transaction Manager

Name	`SecureWriteTransactionManager`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Extends	`WriteTransactionManager`
Since	0.7
Purpose	**Synchronous** write transaction manager that requires a control SAM. Used to write SAM parameters, transfer keys and transfer lock files. An instance is obtained via `LegacySamApiFactory.createSecureWriteTransactionManager`.

Prepare Plain Write Lock

Signature	`preparePlainWriteLock(`
	`  lockIndex: Byte,`
	`  lockParameters: Byte,`
	`  lockValue: ByteArray`
	`) → Self`

Since	0.7
Description	Schedules a **Write Key** command to set the lock file with a plain-text 16-byte lock value.
Parameters	`lockIndex` — the index of the lock file. `lockParameters` — the lock permissions parameters. `lockValue` — a 16-byte byte array representing the lock's value.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — `lockValue` is within range.
Errors	None.

Prepare Transfer Lock

Signature	`prepareTransferLock(`
	`  lockIndex: Byte,`
	`  lockParameters: Byte`
	`) → Self`

Since	0.7
Description	Schedules a **Write Key** command transferring a lock file from the control SAM to the target SAM.
Parameters	`lockIndex` — the index of the lock file. `lockParameters` — the lock permissions parameters.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

Prepare Transfer Lock Diversified

Signature	`prepareTransferLockDiversified(`
	`  lockIndex: Byte,`
	`  lockParameters: Byte`
	`) → Self`

Since	0.7
Description	Schedules a **Write Key** command transferring a lock file from the control SAM to the target SAM, with diversification by the target SAM serial number.
Parameters	`lockIndex` — the index of the lock file. `lockParameters` — the lock permissions parameters.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

Prepare Transfer System Key

Signature	`prepareTransferSystemKey(`
	`  systemKeyType: SystemKeyType,`
	`  kvc: Byte,`
	`  systemKeyParameters: ByteArray`
	`) → Self`

Since	0.7
Description	Schedules a **Write Key** command transferring a system key from the control SAM to the target SAM. `systemKeyParameters` MUST be 29 bytes long.
Parameters	`systemKeyType` (`SystemKeyType`) — the system key to transfer. `kvc` — the KVC of the key. `systemKeyParameters` — the 29-byte key parameters.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — every argument is within range.
Errors	None.

Prepare Transfer System Key Diversified

Signature	`prepareTransferSystemKeyDiversified(`
	`  systemKeyType: SystemKeyType,`
	`  kvc: Byte,`
	`  systemKeyParameters: ByteArray`
	`) → Self`

Since	0.7
Description	Schedules a **Write Key** command transferring a system key from the control SAM to the target SAM, with the key first diversified with the target SAM serial number. `systemKeyParameters` MUST be 29 bytes long.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Parameters	`systemKeyType` (`SystemKeyType`) — the system key to transfer. `kvc` — the KVC of the key. `systemKeyParameters` — the 29-byte key parameters.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

Prepare Transfer Work Key

Signature	`prepareTransferWorkKey(`
	`  kif: Byte,`
	`  kvc: Byte,`
	`  workKeyParameters: ByteArray,`
	`  targetRecordNumber: Int`
	`) → Self`

Since	0.7
Description	Schedules a **Write Key** command transferring a work key from the control SAM to the target SAM. `workKeyParameters` MUST be 29 bytes long. `targetRecordNumber` MUST be in `[0..126]`; `0` means the SAM MUST choose the location.
Parameters	`kif` — the KIF of the key. `kvc` — the KVC of the key. `workKeyParameters` — a 29-byte byte array containing the key parameter data. `targetRecordNumber` — the number of the record where to store the key (in range [0..126]).
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — every argument is within range.
Errors	None.

Prepare Transfer Work Key Diversified

Signature	`prepareTransferWorkKeyDiversified(`
	`  kif: Byte,`
	`  kvc: Byte,`
	`  workKeyParameters: ByteArray,`
	`  targetRecordNumber: Int`
	`) → Self`

Since	0.7
Description	Schedules a **Write Key** command transferring a work key from the control SAM to the target SAM, with diversification by the target SAM serial number. `workKeyParameters` MUST be 29 bytes long. `targetRecordNumber` MUST be in `[0..126]`; `0` means the SAM MUST choose the location.
Parameters	`kif` — the KIF of the key. `kvc` — the KVC of the key. `workKeyParameters` — a 29-byte byte array containing the key parameter data. `targetRecordNumber` — the number of the record where to store the key (in range [0..126]).
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

Prepare Transfer Work Key Diversified With Specific Diversifier

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Signature	`prepareTransferWorkKeyDiversifiedWithSpecificDiversifier(`
	`  kif: Byte,`
	`  kvc: Byte,`
	`  workKeyParameters: ByteArray,`
	`  targetRecordNumber: Int,`
	`  diversifier: ByteArray`
	`) → Self`

Since	0.7
Description	Schedules a **Write Key** command transferring a work key from the control SAM to the target SAM, with diversification by the supplied 8-byte diversifier. `workKeyParameters` MUST be 29 bytes long. `targetRecordNumber` MUST be in `[0..126]`; `0` means the SAM MUST choose the location.
Parameters	`kif` — the KIF of the key. `kvc` — the KVC of the key. `workKeyParameters` — a 29-byte byte array containing the key parameter data. `targetRecordNumber` — the number of the record where to store the key (in range [0..126]). `diversifier` — a 8-byte byte array to use as key diversifier.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

Prepare Write SAM Parameters

Signature	`prepareWriteSamParameters(`
	`  parameters: ByteArray`
	`) → Self`

Since	0.7
Description	Schedules a **Write Parameters** command. The supplied buffer MUST be 29 bytes long.
Parameters	`parameters` — a 29-byte byte array representing the content of the SAM parameters file.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — the argument is within range.
Errors	None.

5.3.10. Signature Computation Data

Name	`SignatureComputationData`
Kind	Sealed interface
Namespace	`calypso.crypto.legacysam.transaction`
Since	0.1
Purpose	Common supertype of the signature computation criteria. Carries no member; it exists to let an operation accept either form.

Its permitted implementations are `BasicSignatureComputationData` and `TraceableSignatureComputationData`, which carry the criteria of the two command families.

5.3.11. Signature Verification Data

Name	`SignatureVerificationData`
Kind	Sealed interface

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Namespace	`calypso.crypto.legacysam.transaction`
Since	0.1
Purpose	Common supertype of the signature verification criteria. Carries no member; it exists to let an operation accept either form.

Its permitted implementations are `BasicSignatureVerificationData` and `TraceableSignatureVerificationData`, which carry the criteria of the two command families.

5.3.12. Transaction Manager

Name	`TransactionManager`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Extends	`CardTransactionManager` (CNA-TR-API)
Since	0.1
Purpose	Common base of every legacy-SAM transaction manager. Provides command processing for the SAM.

The Legacy SAM `TransactionManager` is not generic; processing of prepared commands is inherited from `CardTransactionManager` (CNA-TR-API) via the `processCommands()` operation. The Legacy SAM `TransactionManager` does **not** redefine `processCommands()`; it relies on the inherited contract. Additional failure modes specific to the SAM dialog MAY be reported through `InvalidSignature` (signature verification failure) and `InconsistentData` (data inconsistency).

5.3.13. Write Transaction Manager

Name	`WriteTransactionManager`
Kind	Interface
Namespace	`calypso.crypto.legacysam.transaction`
Extends	`TransactionManager`
Since	0.2
Purpose	Common base of every **write** transaction manager. Writes MAY be performed synchronously or asynchronously.

Prepare Write Counter Ceiling

Signature	`prepareWriteCounterCeiling(`
	`  counterNumber: Int,`
	`  ceilingValue: Int`
	`) → Self`

Since	0.2
Description	Schedules a **Write Ceilings** command for a single counter. The ceiling value MUST be a positive integer less than or equal to `FFFFFAh`. **Warning:** in an asynchronous transaction, the content of the `LegacySam` object MUST NOT be updated.
Parameters	`counterNumber` — the number of the counter whose ceiling is to be written (in range [0..26]). `ceilingValue` — the desired value for the ceiling. The ceiling value is defined as a positive integer less than or equal to 16777210 (in hexadecimal: FFFFFAh).
Returns	The current instance (`Self`).

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — every argument is within range.
Errors	None.

Prepare Write Counter Configuration

Signature	`prepareWriteCounterConfiguration(`
	`  counterNumber: Int,`
	`  ceilingValue: Int,`
	`  counterIncrementAccess: CounterIncrementAccess`
	`) → Self`

Since	0.2
Description	Schedules a **Write Ceilings** command for a counter ceiling and its free-incrementation configuration. Because the command writes a full record of nine counters, the application MUST have first read the status of the counters of that record (or have called this method for each of the other eight counters of the same record), otherwise an error is raised when processing the commands. **Warning:** in the case of an asynchronous transaction, the content of the `LegacySam` is not updated.
Parameters	`counterNumber` — the number of the counter to configure. `ceilingValue` — the ceiling value to set for the counter. `counterIncrementAccess` (`CounterIncrementAccess`) — the free-incrementation access configuration.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — every argument is within range.
Errors	None.

5.4. SPI Interfaces

5.4.1. Legacy SAM Dynamic Unlock Data Provider SPI

Name	`LegacySamDynamicUnlockDataProviderSpi`
Kind	Interface (SPI)
Namespace	`calypso.crypto.legacysam.spi`
Since	0.4
Purpose	Interface that the application MUST implement to compute the dynamic unlock data expected by the SAM.

Get Unlock Data

Signature	`getUnlockData(`
	`  samSerialNumber: ByteArray,`
	`  samChallenge: ByteArray`
	`) → ByteArray`

Since	0.4
Description	Returns the dynamic unlock data computed by an origin SAM. The serial number and the challenge are required to prepare the **SAM Generate Unlock** command.
Parameters	`samSerialNumber` — the target SAM serial number. `samChallenge` — the challenge provided by the target SAM.
Returns	An 8-byte `ByteArray`.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

5.4.2. Legacy SAM Revocation Service SPI

Name	`LegacySamRevocationServiceSpi`
Kind	Interface (SPI)
Namespace	`calypso.crypto.legacysam.spi`
Since	0.1
Purpose	Interface that the application MUST implement to check dynamically whether a SAM is revoked.

Is SAM Revoked

Signature	`isSamRevoked(`
	`  serialNumber: ByteArray,`
	`  counterValue: Int`
	`) → Boolean`

Since	0.1
Description	Checks if the SAM with the supplied serial number and counter value is revoked. The supplied serial number MAY be complete (4 bytes) or partial (the 3 least significant bytes).
Parameters	`serialNumber` — the complete or partial SAM serial number to check. `counterValue` — the SAM counter value.
Returns	`true` if the SAM is revoked, `false` otherwise.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

5.4.3. Legacy SAM Static Unlock Data Provider SPI

Name	`LegacySamStaticUnlockDataProviderSpi`
Kind	Interface (SPI)
Namespace	`calypso.crypto.legacysam.spi`
Since	0.4
Purpose	Interface that the application MUST implement to compute the static unlock data expected by the target SAM.

Get Unlock Data

Signature	`getUnlockData(samSerialNumber: ByteArray) → ByteArray`
Since	0.4
Description	Returns the static unlock data expected by the target SAM. The serial number MAY be used as a diversifier.
Parameters	`samSerialNumber` — the target SAM serial number.
Returns	A 16-byte `ByteArray`.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

5.5. Enumerations

5.5.1. Counter Increment Access

Name	`CounterIncrementAccess`
Kind	Enumeration
Namespace	`calypso.crypto.legacysam`
Since	0.3
Purpose	Possible access rights for incrementing event counters using the **Increment Counter** command.

Value	Since	Description
`FREE_COUNTING_ENABLED`	0.3	Enables the use of **Increment Counter** on the targeted event counter.
`FREE_COUNTING_DISABLED`	0.3	Forbids the use of **Increment Counter** on the targeted event counter.

5.5.2. Get Data Tag

Name	`GetDataTag`
Kind	Enumeration
Namespace	`calypso.crypto.legacysam`
Since	0.5
Purpose	Output data tags retrievable through the **Get Data** command. MAY NOT be applicable to all products.

Value	Since	Description
`CA_CERTIFICATE`	0.5	CA Certificate (CACert). MAY be unavailable.

5.5.3. Legacy SAM Product Type

Name	`LegacySamProductType`
Kind	Enumeration
Namespace	`calypso.crypto.legacysam.sam`
Since	0.1
Purpose	Product type of a legacy SAM.

Value	Since	Description
`SAM_C1`	0.1	SAM C1.
`HSM_C1`	0.1	SAM C1 HSM.
`SAM_S1E1`	0.1	SAM S1E1.
`SAM_S1DX`	0.1	SAM S1Dx.
`UNKNOWN`	0.1	Unidentified SAM.

5.5.4. SAM Traceability Mode

Name	`SamTraceabilityMode`

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Kind	Enumeration
Namespace	`calypso.crypto.legacysam.transaction`
Since	0.3
Purpose	SAM traceability mode used with traceable signature operations.

Value	Since	Description
`FULL_SERIAL_NUMBER`	0.3	Full SAM serial number (4 bytes).
`TRUNCATED_SERIAL_NUMBER`	0.3	Truncated SAM serial number (3 LSBytes).

5.5.5. System Key Type

Name	`SystemKeyType`
Kind	Enumeration
Namespace	`calypso.crypto.legacysam`
Since	0.2
Purpose	System key types of a legacy SAM. Each type corresponds to a specific role.

Value	Since	Description
`PERSONALIZATION`	0.2	Personalisation key—deciphers and authorises the writing of parameters and system keys.
`KEY_MANAGEMENT`	0.2	Work-file key—deciphers and authorises the writing of work keys.
`RELOADING`	0.2	Reloading key—deciphers and authorises the writing of counter ceilings.
`AUTHENTICATION`	0.2	Authentication key—generates the signature of data read from the SAM.

5.6. Errors

Every error defined below carries two properties: `message` (`String`), which describes the condition, and `cause` (`Any?`), the underlying error that triggered it, `null` when there is none. They are not restated for each error.

5.6.1. Inconsistent Data

Name	`InconsistentData`
Kind	Error
Namespace	`calypso.crypto.legacysam.transaction`
Since	0.1
Purpose	Indicates the detection of inconsistent data.

The inconsistency falls into one of the following cases:

- a **de-synchronisation of the APDU exchanges**, that is, a number of APDU responses different from the number of APDU requests;
- an **inconsistency in the card data**, which can happen for example when the data read outside the secure session differs from the data read inside it.

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

5.6.2. Invalid Signature

Name	`InvalidSignature`
Kind	Error
Namespace	`calypso.crypto.legacysam.transaction`
Since	0.1
Purpose	Indicates that a signature is invalid.

5.6.3. SAM Revoked

Name	`SamRevoked`
Kind	Error
Namespace	`calypso.crypto.legacysam.transaction`
Since	0.1
Purpose	Indicates that the SAM is revoked.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement